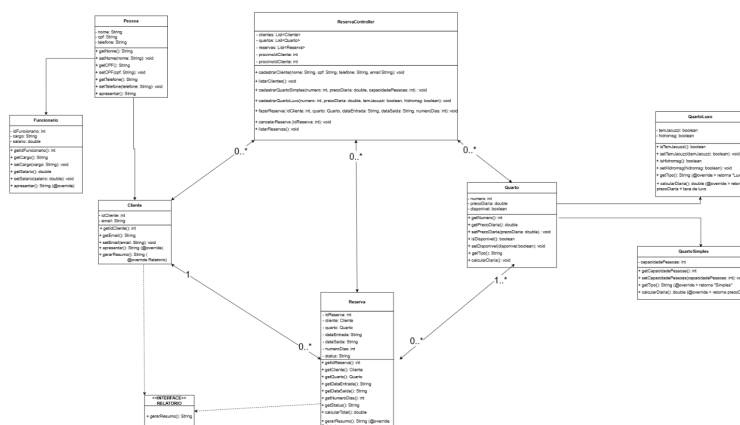


Sistema de Reservas de HOTEL/POUSADA

Requisito	O que foi feito
1 classe abstrata ou superclasse	Pessoa e Quarto
2 ou mais subclasses	P -> Cliente e Funcionar / Q -> QuartoSimples e QuartoLuxo
1 interface	Falta codar, porém já há o <<Interface>> Relatorio com gerarResumo():String, implementada por Cliente e Reserva
Atributos privados com get/set	Todas as classes possuem atributos private e métodos de getters e setters
1 uso real de polimorfismo	Estruturas prontas no controller (List<Quarto>, List<Pessoa>, List<Reserva
Separação em pacotes MVC	Model e Controller já prontas no diagrama, falta implementá-las junto com a classe MenuView(View) em código
Diagrama de classes UML	Completo, com herança, interface, associações e cardinalidades

Diagrama de Classes UML



Encapsulamento: Atributos com modificador private e métodos públicos de acesso.

- Getters: getNome(), getCpf(), getPrecoDiaria(), getStatus(), etc.
- Setters: setNome(), setPrecoDiaria(), etc.
- Método de negócio: calcularTotal() em Reserva (numeroDias x quarto.calcularDiaria()).

Herança: Relação “é um” aplicada em duas hierarquias/classes:

- Cliente É UMA Pessoa | Funcionário É UMA Pessoa
- QuartoSimples É UM Quarto | QuartoLuxo É UM Quarto

Métodos sobrescritos com @Override:

- apresentar() - Em Cliente e Funcionário (retorna informações formatadas de cada tipo).
- getTipo() - Em QuartoSimples (retorna 'Simples') e QuartoLuxo (retorna 'Luxo')
- calcularDiaria() - Em QuartoSimples (retorna precoDiaria) e QuartoLuxo (retorna precoDiaria + taxa de luxo).

Interface:

- Interface criada: Relatorio
- Método gerarResumo(): String - Implementada por Cliente e Reserva
- Justificativa: ambas as classes têm a capacidade de gerar um resumo textual dos seus dados, representando uma capacidade comum entre classes de hierarquias diferentes

src/

```
├── model/
│   ├── Pessoa.java (abstrata)
│   ├── Cliente.java (extends Pessoa, implements Relatorio)
│   ├── Funcionario.java (extends Pessoa)
│   ├── Quarto.java (abstrata)
│   ├── QuartoSimples.java (extends Quarto)
│   ├── QuartoLuxo.java (extends Quarto)
│   ├── Reserva.java (implements Relatorio)
│   └── Relatorio.java (interface)
├── view/
│   └── MenuView.java
├── controller/
│   └── ReservaController.java
└── Main.java
```

Guilherme	Model — Negócio + Interface	Quarto (abstrata), QuartoSimples, QuartoLuxo, Reserva. Interface Relatorio. Implementar gerarResumo() e calcularTotal().	❑ Falta código
Eduarda	Controller + Main	ReservaController com todos os métodos. Demonstrar polimorfismo com List<Quarto> e List<Pessoa>. Classe Main.	❑ Falta código
Ana Letícia	View + Apresentação	Classe MenuView com Scanner. Slides dos tópicos 4.1, 4.3, 4.4, 4.5, 4.6, 4.7, 4.8 e 4.9.	❑ Falta código